JSS Mahavidyapeetha

JSS COLLEGE OF ARTS, COMMERCE & SCIENCE

M.Sc. - Computer Science

III Semester Examination, July 2025

.Net Technologies

Time: 3 Hours | Max. Marks: 70

Part A

Answer any Five questions.

Q. Mention Any Two Type-Safe Measures. (2 Marks)

📝 **Answer:**

Two type-safe measures in C# are:

1. **Common Type System (CTS)** – Ensures all .NET languages use common data types.
2. **Common Language Specification (CLS)** – Defines rules for language interoperability and type safety.

Q. Expand CLR and CTS. (2 Marks)

📝 **Answer:**

- **CLR → Common Language Runtime**
- **CTS → Common Type System**

📌 CLR manages the execution of .NET programs, while CTS defines the common data types used by all .NET languages. 💯📚

Q. What are Arrays? Write its Merits. (2 Marks)

📝 **Answer:**

An **Array** is a collection of elements of the same data type stored in contiguous memory locations and accessed using an index.

Merits:

1. Stores multiple values using a single variable.
2. Provides fast and easy access to data using indexes.

Bro, looking at the question paper image:

Q. Write the Significance of ::. (2 Marks)

📝 **Answer:**

:: is the **Scope Resolution Operator** in C#.

It is used to access a member of a namespace and resolve ambiguity when two identifiers have the same name.

Example:

System::Console.WriteLine("Hello");

📌 **Significance:** Helps identify the exact namespace or scope to which a member belongs.

Q. Define Static Class. Write its Syntax. (2 Marks)

📝 **Answer:**

A **Static Class** is a class that cannot be instantiated (objects cannot be created) and contains only static members.

Syntax

```
static class ClassName
{
    // Static members
```

}

Example:

```
static class MathDemo
{
   public static int x = 10;
}
```

📌 A static class is accessed using the **class name** and not through objects. 💯📚

Q. What is Abstract Class? Mention its Types. (2 Marks)

📝 **Answer:**

An **Abstract Class** is a class declared using the **abstract** keyword that cannot be instantiated and is used as a base class for other classes.

Types of Methods in an Abstract Class

1. **Abstract Methods** – Methods without implementation.
2. **Concrete (Non-Abstract) Methods** – Methods with implementation.

Syntax:

```
abstract class Shape
{
   public abstract void Draw();
}
```

📌 An abstract class can contain both **abstract** and **non-abstract** methods. 💯📚

Q. Mention Merits of MDI. (2 Marks)

📝 **Answer:**

Merits of MDI (Multiple Document Interface):

1. Allows multiple documents/forms to be opened in a single window.

2. Improves organization and management of documents.
3. Reduces desktop clutter.
4. Provides easy navigation between child windows.

Q. List Out Applications of ADO.NET. (2 Marks)

📝 **Answer:**

Applications of ADO.NET:

1. Database connectivity in .NET applications.
2. Retrieving and updating data from databases.
3. Developing data-driven Windows and Web applications.
4. Accessing SQL Server, Oracle, MS Access, and XML data sources.

Part-B

Answer all the questions.

Q. Write a Note on Components of .NET Framework. (8 Marks)

📝 **Answer:**

📌 **Introduction**

The **.NET Framework** consists of several components that work together to develop and execute .NET applications efficiently.

1️⃣ **Common Language Runtime (CLR)**

- Execution engine of .NET.
- Manages program execution.
- Provides:
    - Memory Management

- Garbage Collection
- Security
- Exception Handling
- JIT Compilation

2️⃣ Framework Class Library (FCL)

- Collection of reusable classes and methods.
- Used for:
  - File Handling
  - Database Access
  - Networking
  - GUI Development

Examples:

System

System.IO

System.Data

3️⃣ Common Type System (CTS)

- Defines all data types in .NET.
- Ensures type safety.
- Enables language interoperability.

Examples:

int

float

string

char

---

4 Common Language Specification (CLS)

- Set of rules that all .NET languages must follow.
- Allows interaction between different .NET languages.

---

5 Microsoft Intermediate Language (MSIL)

- Intermediate code generated by .NET compilers.
- Independent of machine architecture.
- Executed by CLR.

---

6 Just-In-Time Compiler (JIT)

- Converts MSIL into machine code at runtime.
- Improves execution efficiency.

---

7 Assemblies

- Basic unit of deployment in .NET.
- Contains:
    - MSIL
    - Metadata
    - Manifest

Types:

- Private Assembly
- Shared Assembly

---

8 Metadata

- Information about program components.

- Describes classes, methods, properties, and data types.
- Used by CLR during execution.

📌 **Advantages of .NET Components**

✅ Language interoperability

✅ Type safety

✅ Security

✅ Automatic memory management

✅ Reusable libraries

Q. Explain the Following: (8 Marks)

i) Metadata

ii) C# Preprocessor Directives

📝 **Answer:**

i) Metadata

Definition

Metadata is information about a program stored within an assembly. It describes the structure of the program such as classes, methods, properties, events, and data types.

Metadata is often called **"Data about Data."**

Functions of Metadata

- Stores information about classes and methods.
- Used by CLR during execution.
- Supports reflection.
- Eliminates the need for separate type libraries.

Example

Metadata contains information such as:

```
class Student
{
   int Id;
   string Name;
}
```

The details of Student, Id, and Name are stored as metadata.

Advantages

- ✅ Provides type information
- ✅ Supports runtime execution
- ✅ Helps in code inspection
- ✅ Enables language interoperability

---

ii) C# Preprocessor Directives

Definition

Preprocessor Directives are commands processed before the actual compilation of a C# program.

They begin with the # symbol.

Common Preprocessor Directives

1. #define

Defines a symbol.

```
#define TEST
```

---

2. #undef

Removes a previously defined symbol.

#undef TEST

---

3. #if

Executes code if a condition is true.

#define TEST

#if TEST

Console.WriteLine("Testing");

#endif

---

4. #else

Provides an alternate block.

#if TEST

Console.WriteLine("True");

#else

Console.WriteLine("False");

#endif

---

5. #region and #endregion

Used to create collapsible code sections.

#region Variables

int x = 10;

#endregion

---

6. #error

Generates a compilation error.

#error Invalid Program

---

7. #warning

Displays a warning message.

#warning Check this code

Advantages of Preprocessor Directives

✅ Conditional compilation

✅ Improves code organization

✅ Helps debugging

✅ Provides compile-time control

Q. Write a Note on Identifiers and Keywords. (6 Marks)

📝 **Answer:**

1) Identifiers

Definition

Identifiers are user-defined names given to program elements such as variables, methods, classes, objects, and namespaces.

Rules for Identifiers

- Must begin with a letter or underscore (_).
- Cannot start with a digit.
- Cannot contain special characters except _.
- Cannot be a keyword.
- C# is case-sensitive.

Examples

int age;

string Name;

class Student

Valid Identifiers

student

_marks

TotalAmount

Invalid Identifiers

1student

class

name#

---

2) Keywords

Definition

Keywords are reserved words that have predefined meanings in C# and cannot be used as identifiers.

Examples of Keywords

int

float

class

public

private

if

else

while

for

return

Example

```
public class Student
{
   int age;
}
```

Here, public, class, and int are keywords.

Difference Between Identifiers and Keywords

Identifiers	**Keywords**
User-defined names	Reserved words
Can be created by programmer	Predefined by C#
Used to identify program elements	Have special meaning
Example: age, Student	Example: int, class, public

Q. Explain Type Conversions, Boxing and Unboxing. (8 Marks)

📝 **Answer:**

📌 **Introduction**

Type Conversion is the process of converting data from one type to another. C# supports automatic and manual conversions. Boxing and Unboxing are special type conversions between value types and reference types.

---

1) Type Conversion

Definition

Type Conversion is the process of changing a value from one data type to another.

Types of Type Conversion

A) Implicit Conversion

- Performed automatically by the compiler.
- No data loss occurs.
- Smaller type → Larger type.

Example

int a = 100;

double b = a;

B) Explicit Conversion (Casting)

- Performed manually by the programmer.
- Larger type → Smaller type.
- Data loss may occur.

Example

double x = 25.75;

int y = (int)x;

Output:

25

C) Using Convert Class

Used to convert one type to another safely.

Example

string s = "100";

int n = Convert.ToInt32(s);

---

2) Boxing

Definition

Boxing is the process of converting a **value type** into a **reference type (object)**.

Example

int x = 10;

object obj = x;

Here, the integer value 10 is converted into an object.

Advantages

- Converts value types into objects.
- Allows value types to be stored in object references.

---

3) Unboxing

Definition

Unboxing is the process of converting a **reference type (object)** back into a **value type**.

Example

object obj = 10;

int x = (int)obj;

Here, the object is converted back into an integer.

Advantages

- Retrieves the original value type from an object.
- Enables operations on the original data type.

Difference Between Boxing and Unboxing

Boxing	**Unboxing**
Value Type → Object	Object → Value Type

Boxing	Unboxing
Automatic conversion	Explicit conversion
Stores value in heap	Retrieves value from heap
Increases memory usage	Restores original value

✅ Conclusion

Type Conversion allows data to be converted between different types. It can be **implicit**, **explicit**, or performed using the **Convert class**. **Boxing** converts a value type into an object, whereas **Unboxing** converts an object back into a value type. These mechanisms provide flexibility in handling data within C# programs.

Q. Write a Note on Scope Resolution Operator. (4 Marks)

📝 Answer:

📌 Definition

The **Scope Resolution Operator (::)** is used to access members of a namespace and to resolve ambiguity when two identifiers have the same name.

It specifies the exact scope to which a member belongs.

📌 Syntax

Namespace::Member

📌 Example

System::Console.WriteLine("Hello");

Here, Console is explicitly accessed from the System namespace.

📌 **Uses**

- Accesses namespace members.
- Resolves naming conflicts.
- Identifies the exact scope of an identifier.

Q. Explain Various Types of Arrays with Example. (8 Marks)

📝 **Answer:**

📌 **Introduction**

An **Array** is a collection of elements of the same data type stored in contiguous memory locations.

C# supports different types of arrays to store and manage data efficiently.

---

1) Single-Dimensional Array

Definition

A single-dimensional array stores elements in a single row.

Example

```
int[] arr = {10,20,30,40,50};


Console.WriteLine(arr[2]);
```

Output

30

---

2) Multi-Dimensional Array

Definition

A multi-dimensional array stores data in rows and columns.

Example

```
int[,] arr =
{
   {1,2},
   {3,4}
};


Console.WriteLine(arr[1,0]);
```

Output

3

---

3) Jagged Array

Definition

A **Jagged Array** is an array of arrays where each row can have a different number of elements.

Example

```
int[][] arr = new int[3][];


arr[0] = new int[] {1,2};
arr[1] = new int[] {3,4,5};
arr[2] = new int[] {6};
```

Features

- Rows can have different sizes.
- Saves memory.

---

4) Dynamic Array (Using Array Class)

Definition

An array whose size can be managed dynamically using .NET collection classes.

Example

```
ArrayList list = new ArrayList();


list.Add(10);

list.Add(20);
```

Difference Between Array Types

Type	Description
Single-Dimensional	One row of elements
Multi-Dimensional	Rows and columns
Jagged Array	Array of arrays with different row sizes
Dynamic Array	Size can grow dynamically

📌 **Advantages of Arrays**

✅ Stores multiple values using one variable

✅ Easy access using index

✅ Efficient memory usage

✅ Simplifies data management

Q. How do you Declare Partial Class and Methods? Explain with an Example. (8 Marks)

📝 **Answer:**

📌 Introduction

C# provides **Partial Classes** and **Partial Methods** to split the definition of a class into multiple files. This improves code organization and maintainability.

---

1) Partial Class

Definition

A **Partial Class** allows the definition of a class to be divided into two or more files using the partial keyword.

At compile time, all parts are combined into a single class.

Syntax

```
partial class ClassName
{
   // Members
}
```

Example of Partial Class

File1.cs

```
partial class Student
{
   public void Display()
   {
      Console.WriteLine("Student Details");
   }
}
```

File2.cs

```
partial class Student
{
```

```
    public void Show()
    {
        Console.WriteLine("Student Information");
    }
}
```

Main Program

```
using System;


class Program
{
    static void Main()
    {
        Student s = new Student();


        s.Display();
        s.Show();
    }
}
```

Output

```
Student Details
Student Information
```

---

2) Partial Method

Definition

A **Partial Method** is a method declared in one part of a partial class and implemented in another part of the same partial class.

It must be declared using the partial keyword.

Syntax

partial void MethodName();

Example of Partial Method

```
using System;


partial class Demo
{
   partial void ShowMessage();

   public void Display()
   {
      ShowMessage();
   }
}

partial class Demo
{
   partial void ShowMessage()
   {
      Console.WriteLine("Partial Method Executed");
   }
}

class Program
```

```
{
   static void Main()
   {
      Demo d = new Demo();
      d.Display();
   }
}
```

Output

Partial Method Executed

Advantages of Partial Classes and Methods

- ✅ Improves code organization
- ✅ Multiple developers can work on the same class
- ✅ Simplifies maintenance
- ✅ Useful in large projects
- ✅ Supports code generation tools

Difference Between Partial Class and Partial Method

Partial Class	**Partial Method**
Splits a class into multiple files	Splits method declaration and implementation
Declared using partial class	Declared using partial void
Improves code organization	Improves modularity

Partial Class	Partial Method
Combines at compile time	Implemented within partial classes

Q. Explain Syntax of Struct with a Suitable Example. (8 Marks)

📝 **Answer:**

📌 **Introduction**

A **Struct (Structure)** is a user-defined value type in C# used to group related data members and methods into a single unit.

Structs are similar to classes, but they are **value types** and are stored in stack memory.

📌 **Definition**

A **Struct** is a value type that can contain variables, methods, constructors, properties, and indexers.

It is declared using the **struct** keyword.

📌 **Syntax of Struct**

```
struct StructName
{
    // Data Members

    // Methods
}
```

📌 **Example Program**

```
using System;
```

```
struct Student
{
    public int Id;
    public string Name;

    public void Display()
    {
        Console.WriteLine("Id = " + Id);
        Console.WriteLine("Name = " + Name);
    }
}

class Program
{
    static void Main()
    {
        Student s;

        s.Id = 101;
        s.Name = "Ram";

        s.Display(); }  }
```

📌 **Output**

```
Id = 101
```

Name = Ram

📌 **Features of Struct**

- Struct is a value type.
- Declared using struct keyword.
- Can contain fields, methods, properties, and constructors.
- Supports interfaces.
- Stored in stack memory.

📌 **Advantages of Struct**

✅ Faster than classes for small objects

✅ Efficient memory usage

✅ Suitable for storing simple data

✅ Supports encapsulation

📌 **Difference Between Struct and Class**

Struct	**Class**
Value Type	Reference Type
Stored in Stack	Stored in Heap
Cannot support inheritance	Supports inheritance
Faster for small data	Suitable for large objects

Q. Explain Any Four String Handling Operations with Their Syntax. (7 Marks)

📝 **Answer:**

📌 Introduction

A **String** is a sequence of characters enclosed within double quotes. C# provides various string handling methods to manipulate strings.

---

1) ToUpper()

Purpose

Converts all characters of a string into uppercase.

Syntax

string.ToUpper();

Example

string s = "hello";

Console.WriteLine(s.ToUpper());

Output

HELLO

---

2) ToLower()

Purpose

Converts all characters of a string into lowercase.

Syntax

string.ToLower();

Example

string s = "HELLO";

Console.WriteLine(s.ToLower());

Output

hello

---

3) Length

Purpose

Returns the number of characters in a string.

Syntax

string.Length

Example

string s = "Computer";

Console.WriteLine(s.Length);

Output

8

---

4) Replace()

Purpose

Replaces a specified character or string with another character or string.

Syntax

string.Replace(oldValue,newValue);

Example

string s = "Java";

Console.WriteLine(s.Replace("Java","C#"));

Output

C#

📌 **Other String Methods**

- Substring()
- Trim()
- Concat()

- Contains()
- StartsWith()
- EndsWith()

Q. Explain Polymorphism in Detail. (8 Marks)

📝 **Answer:**

📌 **Introduction**

Polymorphism is one of the fundamental concepts of Object-Oriented Programming (OOP).

The word **Polymorphism** means **"Many Forms"**.

It allows the same method or object to perform different tasks in different situations.

📌 **Definition**

Polymorphism is the ability of a method, object, or operator to take many forms and perform different actions based on the context.

📌 **Types of Polymorphism**

1️⃣ **Compile-Time Polymorphism**

(Method Overloading)

2️⃣ **Runtime Polymorphism**

(Method Overriding)

---

1) Compile-Time Polymorphism

Definition

Compile-Time Polymorphism is achieved through **Method Overloading**.

Multiple methods have the same name but different parameter lists.

The method to be called is decided during compilation.

Example

```
using System;


class Demo
{
   public void Add(int a, int b)
   {
      Console.WriteLine(a + b);
   }


   public void Add(int a, int b, int c)
   {
      Console.WriteLine(a + b + c);
   }
}


class Program
{
   static void Main()
   {
      Demo d = new Demo();


      d.Add(10,20);
      d.Add(10,20,30);
   }
```

```
}
```

Output

30

60

---

2) Runtime Polymorphism

Definition

Runtime Polymorphism is achieved through **Method Overriding**.

A derived class provides its own implementation of a base class method.

The method to be executed is determined at runtime.

Example

```
using System;

class Animal
{
    public virtual void Sound()
    {
        Console.WriteLine("Animal Sound");
    }
}

class Dog : Animal
{
    public override void Sound()
    {
        Console.WriteLine("Dog Barks");
```

```
    }
}

class Program
{
    static void Main()
    {
        Animal a = new Dog();
        a.Sound();
    }
}
```

Output

Dog Barks

📌 Advantages of Polymorphism

- ✅ Increases code reusability
- ✅ Improves flexibility
- ✅ Reduces code complexity
- ✅ Supports dynamic method invocation
- ✅ Easy maintenance of programs

📌 Difference Between Compile-Time and Runtime Polymorphism

Compile-Time Polymorphism	Runtime Polymorphism
Method Overloading	Method Overriding

Compile-Time Polymorphism	Runtime Polymorphism
Decision at compile time	Decision at runtime
Inheritance not required	Inheritance required
Faster execution	Slightly slower
Same class	Base and derived classes

Q. Explain the Following: (7 Marks)

i) Multiple Event Handling

ii) Unchecked Statement

Answer:

i) Multiple Event Handling

Definition

Multiple Event Handling is a technique in which a single event handler method handles multiple events generated by one or more controls.

This reduces code duplication and improves maintainability.

Example

```
using System;
using System.Windows.Forms;


class Demo : Form
{
   Button b1 = new Button();
   Button b2 = new Button();


   public Demo()
   {
```

```
        b1.Text = "Button1";
        b2.Text = "Button2";

        b1.Click += EventHandlerMethod;
        b2.Click += EventHandlerMethod;
    }

    void EventHandlerMethod(object sender, EventArgs e)
    {
        MessageBox.Show("Button Clicked");
    }
}
```

Advantages

- ✅ Reduces code redundancy
- ✅ Easy maintenance
- ✅ Improves code readability

---

ii) Unchecked Statement

Definition

The **unchecked** statement disables overflow checking for arithmetic operations.

If overflow occurs, no exception is generated and the value wraps around.

Syntax

```
unchecked
{
    // statements
}
```

Example

```
using System;


class Program
{
   static void Main()
   {
      unchecked
      {
         byte x = 255;
         x++;


         Console.WriteLine(x);
      }
   }
}
```

Output

0

Here, 255 + 1 exceeds the range of byte, so the value wraps around to 0.

Advantages

✅ Faster execution

✅ Useful when overflow checking is not required

✅ Reduces runtime overhead

Q. Explain Syntax of Implementing Interface with Example. (8 Marks)

📝 **Answer:**

📌 **Introduction**

An **Interface** is a collection of method declarations without implementation. It defines a contract that must be implemented by the class that inherits it.

Interfaces are used to achieve **abstraction** and **multiple inheritance** in C#.

📌 **Syntax of Interface**

```
interface InterfaceName
{
    void MethodName();
}
```

📌 **Syntax of Implementing Interface**

```
class ClassName : InterfaceName
{
    public void MethodName()
    {
        // Implementation
    }
}
```

📌 **Example Program**

```
using System;


interface IShape
{
```

```
    void Draw();
}

class Circle : IShape
{
    public void Draw()
    {
        Console.WriteLine("Drawing Circle");
    }
}

class Program
{
    static void Main()
    {
        Circle c = new Circle();
        c.Draw();
    }
}
```

📌 **Output**

Drawing Circle

📌 **Steps in Interface Implementation**

Step 1

Declare an interface.

```
interface IShape
{
    void Draw();
}
```

Step 2

Implement the interface in a class.

```
class Circle : IShape
```

Step 3

Provide implementation for all interface methods.

```
public void Draw()
{
    Console.WriteLine("Drawing Circle");
}
```

Step 4

Create object and call methods.

```
Circle c = new Circle();
c.Draw();
```

📌 **Features of Interfaces**

✅ Supports abstraction

✅ Supports multiple inheritance

✅ Improves code reusability

✅ Provides loose coupling

✅ Methods are public by default

📌 **Advantages of Interfaces**

- Hides implementation details.
- Allows multiple inheritance.
- Improves flexibility and maintainability.
- Promotes modular programming.

Q. Explain the Following: (7 Marks)

i) Events and Delegates

ii) Constructor

📝 **Answer:**

i) Events and Delegates

Delegate

Definition

A **Delegate** is a type-safe reference that holds the address of a method.

It allows methods to be called indirectly.

Example

```
delegate void MyDelegate();


class Demo
{
   public static void Show()
   {
      Console.WriteLine("Hello");
   }
```

```
    static void Main()
    {
        MyDelegate d = Show;
        d();
    }
}
```

Output

Hello

Event

Definition

An **Event** is a mechanism through which an object notifies other objects when a specific action occurs.

Events are based on delegates.

Example

public event EventHandler Click;

Uses

- Button Click
- Mouse Click
- Key Press
- Form Load

Advantages

✅ Supports event-driven programming

✅ Improves user interaction

✅ Provides loose coupling

---

ii) Constructor

Definition

A **Constructor** is a special method that is automatically called when an object is created.

It has the same name as the class and does not have a return type.

Syntax

```
class Student
{
   public Student()
   {
      // Constructor
   }
}
```

Example

```
using System;

class Student
{
   public Student()
   {
      Console.WriteLine("Constructor Called");
   }
}

class Program
```

```
{
   static void Main()
   {
      Student s = new Student();
   }
}
```

Output

Constructor Called

Features of Constructor

✅ Automatically invoked

✅ Initializes objects

✅ Same name as class

✅ No return type

Q. How do You Create Windows Forms? Explain. (8 Marks)

📝 **Answer:**

📌 **Introduction**

A **Windows Form** is a graphical user interface (GUI) application in .NET used to create desktop applications with controls such as buttons, labels, text boxes, and menus.

📌 **Steps to Create a Windows Forms Application**

Step 1: Start Visual Studio

Open **Microsoft Visual Studio**.

Step 2: Create a New Project

- Click **File → New → Project**
- Select **Windows Forms Application**
- Enter project name
- Click **Create/OK**

Step 3: Design the Form

Use the **Toolbox** to drag and drop controls onto the form.

Examples:

- Label
- TextBox
- Button
- CheckBox
- RadioButton

Step 4: Set Properties

Modify control properties using the **Properties Window**.

Examples:

Name

Text

Size

Color

Font

Step 5: Write Event Handling Code

Double-click a control to create an event handler.

Example:

```
private void button1_Click(object sender, EventArgs e)
{
   MessageBox.Show("Button Clicked");
}
```

Step 6: Build and Run

- Click **Build → Build Solution**
- Press **F5** to execute the application.

📌 **Example Program**

```
using System;
using System.Windows.Forms;

public class Demo : Form
{
   Button btn;

   public Demo()
   {
      btn = new Button();
      btn.Text = "Click Me";

      btn.Click += Btn_Click;

      Controls.Add(btn);
   }
```

```
    private void Btn_Click(object sender, EventArgs e)
    {
        MessageBox.Show("Hello World");
    }

    static void Main()
    {
        Application.Run(new Demo());
    }
}
```

📌 **Features of Windows Forms**

✅ User-friendly GUI

✅ Event-driven programming

✅ Drag-and-drop controls

✅ Easy application development

✅ Rich collection of controls

📌 **Applications of Windows Forms**

- Student Management System
- Banking Applications
- Inventory Management
- Payroll Systems
- Database Applications

Q. Explain the Following: (7 Marks)

i) Control Properties

ii) Layouts

Answer:

i) Control Properties

Definition

Control Properties are the attributes of a control that determine its appearance, behavior, and functionality in a Windows Forms application.

Common Control Properties

Property	**Description**
Name	Identifies the control
Text	Displays text on the control
Size	Specifies width and height
Location	Specifies position on the form
BackColor	Sets background color
ForeColor	Sets text color
Font	Sets font style and size
Visible	Shows or hides control
Enabled	Enables or disables control

Example

```
button1.Text = "Save";

button1.BackColor = Color.Blue;
```

ii) Layouts

Definition

Layouts determine how controls are arranged and organized on a Windows Form.

They help create a user-friendly and well-structured interface.

Types of Layouts

1. FlowLayoutPanel

- Arranges controls horizontally or vertically.
- Automatically adjusts positions.

2. TableLayoutPanel

- Arranges controls in rows and columns.
- Similar to a table structure.

3. Dock Layout

- Attaches controls to the edges of the form.

4. Anchor Layout

- Maintains the position of controls when the form is resized.

Example

button1.Dock = DockStyle.Top;

📌 **Advantages**

Control Properties

✅ Customize appearance

✅ Control behavior

✅ Improve user interaction

Layouts

✅ Better arrangement of controls

✅ Responsive interface

✅ Easy form design

Q. Explain Event-Handling Mechanism. (8 Marks)

📝 **Answer:**

📌 **Introduction**

Event Handling is the process of responding to events generated by user actions such as button clicks, key presses, mouse movements, etc.

It is the foundation of **event-driven programming** in Windows Forms.

📌 **Definition**

An **Event** is an action performed by the user or system, and an **Event Handler** is a method that responds to that event.

📌 **Components of Event Handling**

1. Event Source

The control that generates the event.

Examples:

- Button
- TextBox
- CheckBox

2. Event

The action performed.

Examples:

- Click
- KeyPress

- MouseMove

3. Event Handler

The method that executes when the event occurs.

📌 Event Registration

An event handler is attached to an event using the += operator.

Syntax

object.Event += new EventHandler(MethodName);

Example

button1.Click += new EventHandler(ButtonClick);

📌 Example Program

```
using System;
using System.Windows.Forms;

class Demo : Form
{
   Button btn;

   public Demo()
   {
      btn = new Button();
      btn.Text = "Click Me";

      btn.Click += new EventHandler(ButtonClick);
```

```
        Controls.Add(btn);
    }

    void ButtonClick(object sender, EventArgs e)
    {
        MessageBox.Show("Button Clicked");
    }

    static void Main()
    {
        Application.Run(new Demo());
    }
}
```

📌 **Output**

Button Clicked

(when the button is clicked)

📌 **Steps in Event Handling Mechanism**

1. User performs an action.
2. Control generates an event.
3. Event is raised.
4. Registered event handler is invoked.
5. Required action is performed.

📌 **Advantages of Event Handling**

✅ Supports event-driven programming

✅ Improves user interaction

✅ Easy handling of user actions

✅ Makes applications responsive

✅ Reduces code complexity

Q. Write a Note on: (7 Marks)

i) Tree View Control

ii) Group Boxes

📝 **Answer:**

i) Tree View Control

Definition

A **TreeView Control** is used to display data in a hierarchical structure consisting of parent and child nodes.

It is commonly used to represent folders, files, and organizational structures.

Features

- Displays data as a tree structure.
- Supports parent-child relationships.
- Allows expanding and collapsing nodes.

Example

```
TreeView tv = new TreeView();


tv.Nodes.Add("Computer");

tv.Nodes[0].Nodes.Add("C Drive");

tv.Nodes[0].Nodes.Add("D Drive");
```

Applications

- Windows Explorer
- Directory structures
- Organization charts

Advantages

✅ Easy navigation

✅ Displays hierarchical data clearly

✅ Supports expandable nodes

---

ii) Group Boxes

Definition

A **GroupBox** is a container control used to group related controls together within a form.

It displays a border and a caption around the grouped controls.

Features

- Organizes related controls.
- Improves form appearance.
- Commonly used with Radio Buttons and Check Boxes.

Example

GroupBox grp = new GroupBox();

grp.Text = "Personal Details";

Applications

- Gender selection
- User information forms
- Settings panels

Advantages

- ✅ Better organization of controls
- ✅ Improves user interface design
- ✅ Makes forms easier to understand
